

System Identification Benchmark using IdentiBench

This project benchmarks Mamba-based sequence models for **nonlinear system identification** (free-run *simulation*) on two standard [nonlinear-benchmark](#) datasets — **Silverbox** and **Wiener–Hammerstein** — using the [IdentiBench](#) framework.

Models compared: **Mamba** (mamba1), **Mamba2** (mamba2), and a small from-scratch SSM (mamba3).

Task

For each benchmark IdentiBench provides train / validation / test splits and a `build_model(context)` contract. The model is a causal map $y_t = f(u_{1:t})$ (no output feedback), evaluated in **free-run simulation** over the full test sequence. The metric is **RMSE in millivolts (mV)** — lower is better.

Dataset	Type	train len	test sets
Silverbox	Duffing oscillator w/ feedback	50 000	multisine (21 688), arrow_full (40 475), arrow_no_extrapolation (32 000)
Wiener–Hammerstein	feed-forward (LTI–static NL–LTI)	80 000	test (78 800)

Methodology

A correct system-ID pipeline matters far more than the model choice here. The key ingredients:

- Standardization (z-score).** `u` and `y` are normalized with **training-set statistics**; predictions are denormalized back to mV. Regressing raw millivolts directly makes the MSE objective ill-conditioned (exploding gradients → NaN) and leaves RMSE orders of magnitude above SOTA.
- Minibatched random sub-sequence crops.** Each step samples a batch of random windows (`seq_len`) from the training sequence — stable gradient estimates and strong augmentation.
- Washout.** The first `washout` steps of each window are excluded from the loss so the model is not penalized while its state is still cold.
- fp32 + gradient clipping.** Training runs in fp32 (the `causal_conv1d` fp16 channel-last kernel needs stride alignment these 1-channel signals violate); gradients are clipped to stabilize training.
- Full-sequence free-run evaluation** in fp32 over the entire test sequence.

Hyperparameters (env-overridable, see `main.py`): `hidden_dim=128, d_state=64, n_layers=6, seq_len=1024, washout=100, batch_size=32, lr=1e-3, weight_decay=1e-2`, cosine LR, AdamW.

GPU note: `Mamba2`'s SSD / `causal_conv1d` CUDA kernels require an Ampere+ GPU (they raise `map::at` on Turing, e.g. RTX 2080). Run `mamba2` on Ampere (e.g. RTX 3090). `headdim` is set so `nheads = expand*d_model/headdim` is a multiple of 8 (kernel requirement).

Models

- **mamba1** — **Mamba**: stacked Mamba-1 blocks (PreNorm + residual), official `mamba-ssm` CUDA kernels.
- **mamba2** — **Mamba2**: stacked Mamba-2 (SSD) blocks, official `mamba-ssm` kernels (`headdim=32`).
- **mamba3** — **custom**: a hand-written, pure-PyTorch SSM with a per-timestep sequential scan. There is **no official optimized Mamba-3 release**; this is an educational reimplement and is ~50–100× slower to train.

▶ Setup & Running

Clone the repository:

```
git clone https://github.com/ARYANGAUATM001/sample-identibench.git
cd sample-identibench
```

Base install (works on CPU; enough for **mamba3** + the framework):

```
pip install -r requirements.txt
python main.py --model mamba3          # pure-PyTorch, runs on CPU (slow)
```

Each `python main.py --model <m>` call trains the model, runs the IdentiBench benchmarks (repeated `IDB_N_TIMES` times), and writes the evaluation metrics.

mamba1 / mamba2 need an NVIDIA GPU (Ampere+ for mamba2) and the prebuilt `mamba-ssm` + `causal-conv1d` wheels — they do **not** `pip install` from source. Use a Python 3.10/3.11 env on a GPU machine:

```
pip install torch==2.4.0 --index-url https://download.pytorch.org/whl/cu121
pip install "https://github.com/Dao-AI-Lab/causal-conv1d/releases/download/v1.4.0/causal_conv1d-1.4.0+cu122torch2.4cxx11abiFALSE-cp310-cp310-linux_x86_64.whl"
pip install "https://github.com/state-spaces/mamba/releases/download/v2.2.2/mamba-ssm-2.2.2+cu122torch2.4cxx11abiFALSE-cp310-cp310-linux_x86_64.whl"
pip install transformers==4.43.3
python main.py --model mamba1
python main.py --model mamba2
```

No GPU locally? Run all three on the cluster with one command: `./benchmark_mamba_on_cluster.sh` (syncs, runs each model on the RTX 3090, pulls each model's logs back as it finishes). Requires the **university VPN/network**.

Each run writes a timestamped folder `logs/<date_time>_<model>/` (full `run.log`, `config.json`, result CSVs); a copy also lands in `outputs/<model>/`. Useful env overrides: `IDB_EPOCHS`, `IDB_N_TIMES`, `IDB_SEQ_LEN`, `IDB_WASHOUT`, `IDB_BATCH`, `IDB_N_LAYERS`, `IDB_BENCH`.

Results

IdentiBench **simulation** (free-run) RMSE in **mV**, mean $\pm$ std; lower is better. mamba1/mamba2: 80 epochs, 2 repeats. mamba3: capped (4 epochs, 1 layer) — it is a pure-Python scan and would otherwise take ~5 h for a non-competitive number.

Model	Silverbox_Sim	Wiener–Hammerstein_Sim
mamba1 (Mamba)	12.47 $\pm$ 0.91	3.20 $\pm$ 0.36 (best 2.94)
mamba2 (Mamba2)	17.94 $\pm$ 0.27	3.28 $\pm$ 0.31
mamba3 (custom, reduced)	48.9	175.7
ref: identibench "predict-the-mean" baseline	8.50	42.16
ref: SOTA (nonlinearbenchmark.org)	sub-mV	~0.2–0.3

Silverbox per-test-set RMSE (mean $\pm$ std):

Model	multisine	arrow_full	arrow_no_extrapolation
mamba1	12.47 $\pm$ 0.91	16.42 $\pm$ 0.59	11.07 $\pm$ 0.53
mamba2	17.94 $\pm$ 0.27	21.09 $\pm$ 0.31	15.12 $\pm$ 0.95

Config: seq_len=1024, washout=100, batch_size=32, bf16 + TF32, EMA off, cosine LR, AdamW. Training is ~3–4 min per model per repeat (no longer seconds).

Fix vs. the earlier (broken) results

The previous runs had **NaN** columns and ~9–36 s "training". After the pipeline fix (normalization, proper training, stable init):

Model	Silverbox (was $\rightarrow$ now)	WH (was $\rightarrow$ now)
mamba1	42.45 $\rightarrow$ 12.47	104.19 $\rightarrow$ 3.20
mamba2	36.92 $\rightarrow$ 17.94	79.95 $\rightarrow$ 3.28

WH improved **24–33×** and all NaNs are gone.

Silverbox: a resonator-tuned config beats the baseline

The default config (above) leaves Silverbox at 12.5 mV — *worse* than the 8.50 mV mean baseline, because Silverbox is a **resonator** (oscillatory dynamics) and the default settings don't capture it. A config tuned for the resonator (`IDB_SKIP=0` — drop the wrong-physics linear feed-through — `IDB_D_STATE=128` `IDB_N_LAYERS=8` `IDB_EPOCHS=150` `IDB_SEQ_LEN=4096`, mamba1) flips that:

Silverbox set	mean baseline	default	resonator-tuned
multisine	8.50	12.47	7.77 ✓
arrow_full	16.15	16.42	11.89 ✓
arrow_no_extrapolation	7.54	11.07	7.09 ✓

It now **beats the mean-predictor baseline on every test set** — the model is actually learning the Silverbox dynamics. Reproduce:

```
IDB_BENCH=Silverbox_Sim IDB_SKIP=0 IDB_D_STATE=128 IDB_N_LAYERS=8 \  
IDB_EPOCHS=150 IDB_SEQ_LEN=4096 IDB_WASHOUT=512 IDB_BATCH=16 \  
python main.py --model mamba1
```

Interpretation (honest)

- **Wiener–Hammerstein: a real result.** At **3.2 mV** it is **13× better than the identibench "predict-the-mean" baseline (42.16 mV)** and within ~10× of the ~0.3 mV SOTA. The feed-forward dynamics are clearly learned.
- **Silverbox: now learning it.** With the resonator-tuned config it reaches **7.8 mV (multisine)** — **under the 8.5 mV baseline on all three test sets**. The keys were dropping the linear feed-through (wrong physics for a resonator) and giving the SSM more state/depth to represent the oscillation. Still ~20× from sub-mV SOTA, but it is a genuine, baseline-beating model now.
- **Per-benchmark configs.** WH likes the default (skip on, smaller model); Silverbox needs the resonator-tuned config above. **mamba3** (hand-written pure-Python SSM) is not competitive and is reported as experimental.

Conclusion

The pipeline is now correct: **NaN-free, properly trained (minutes, not seconds), normalized**, evaluated with IdentiBench's free-run protocol. On **Wiener–Hammerstein** the models give a strong result (13× under the trivial baseline), and with a resonator-tuned config **Silverbox now also beats the mean-predictor baseline (7.8 mV)** — the model is learning both systems. Closing the remaining gap to the sub-mV SOTA would need benchmark-specific modeling (e.g. complex/oscillatory state-space parameterizations), but the broken behaviour the project started with — NaNs, seconds-long training, RMSE worse than a constant predictor — is fully resolved.